

Computational Complexity

Guideline 1

The running time of a comments, declarative and function statements count to zero

Declarative Statements	{	template <class Type>
		class List ;
		int age;
comments	{	//a variable to store age
Function statements	{	void Print();

Total time = zero

Guideline 2

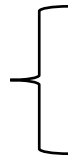
Expressions, memory management & assignment statements for primitive data types have constant running time i.e. one.

Expressions and
assignment statements



```
int a = 4+5;  
char b ='a';
```

Memory Management
statements



```
int * a = new int;  
delete a;
```

Total time = constant

Guideline 3

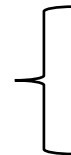
Memory management & assignment statements for objects have constant running time i.e. size of object.

Memory Management
statement of object



Student a,b;
Student * a = new Student;
delete a;

Assignment statement
of object



a = b;

Total time = constant

Guideline 4

Function invocations count as 1 step unless the invocation involves pass by value parameter whose size depends on the instance characteristics i.e. sizes of the value parameters

Total time = constant

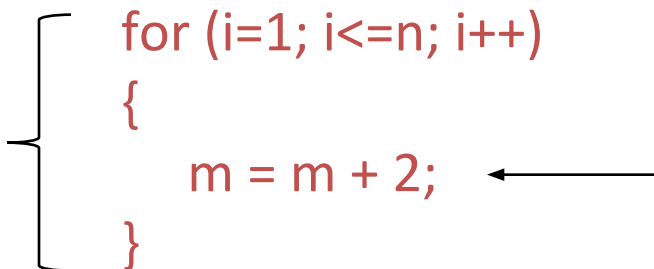
Guideline 5: Loops

The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

executed
 n times

```
for (i=1; i<=n; i++)  
{  
    m = m + 2;  
}
```

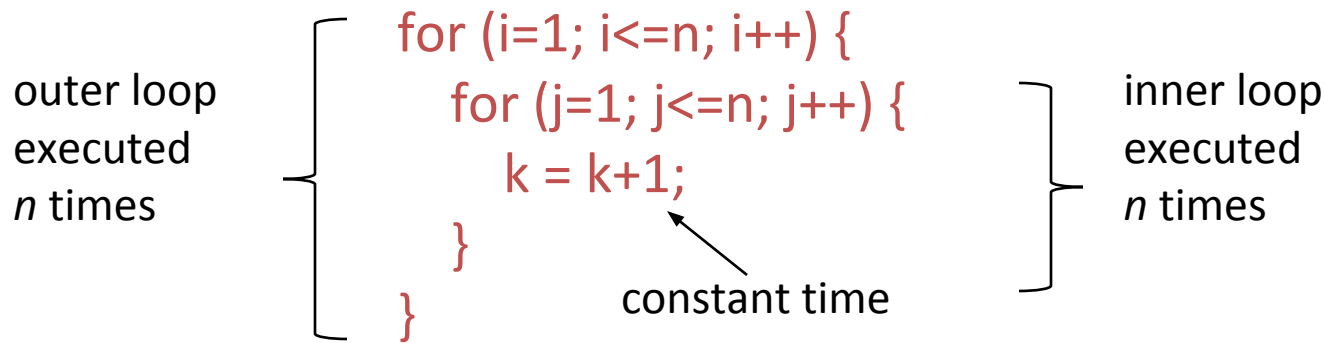
constant time



Total time = (constant) c * n = cn

Guideline 6: Nested loops

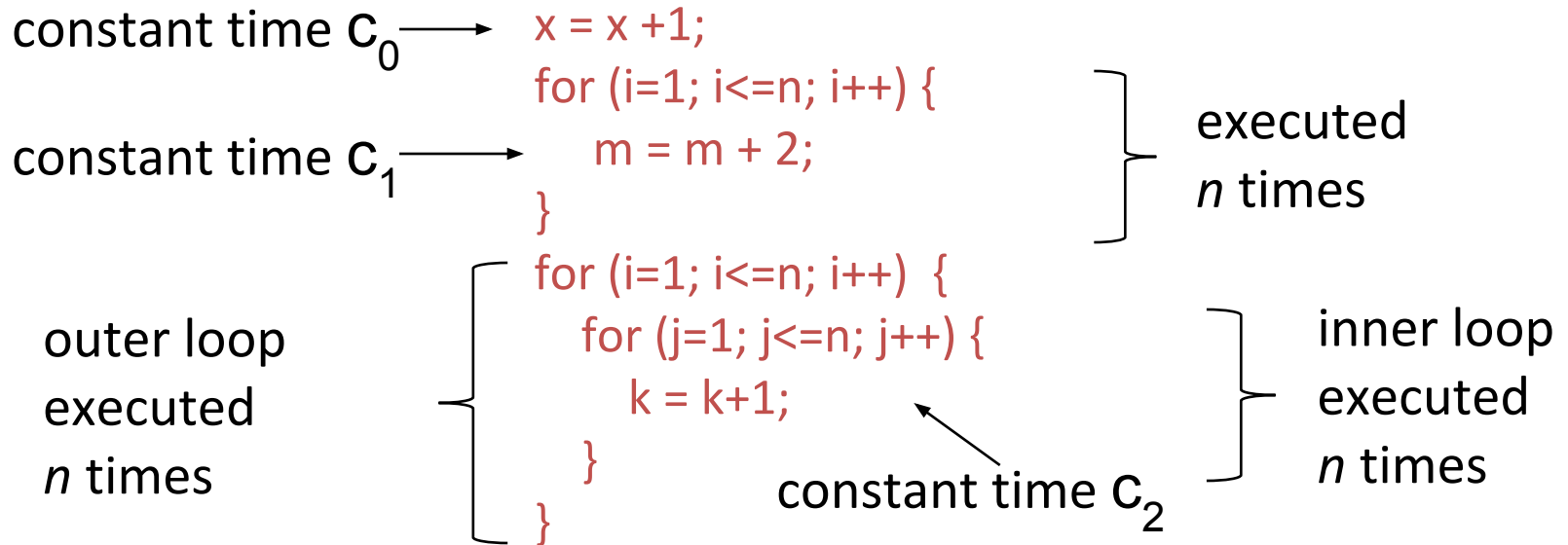
Analyse inside out. Total running time is the product of the sizes of all the loops.



$$\text{Total time} = c * n * n = cn^2$$

Guideline 7: Consecutive statements

Add the time complexities of each statement.



$$\text{Total time} = c_0 + c_1n + c_2n^2$$

Guideline 8: If-then-else statements

```
if( exp)
    statements1;
```

```
else
    statements2;
```

Cost is the number of steps corresponding to exp, statements1 and statements2.

If exp:
Constant c_0

```
if (depth( ) != otherStack.depth( ) ) {
    return false;
}
```

```
else {
    for (int n = 0; n < depth( ); n++) {
        if (!list[n].equals(otherStack.list[n]))
            return false;
    }
}
```

Statements1:
Constant c_1

else part:
(constant c_2 +
Constant c_3) * n

another if :
constant +
constant
(no else part)

$$\text{Total time} = c_0 + c_1 + (c_2 + c_3) * n$$

- The running time of an implementation of the algorithm would depend upon the speed of the computer, programming language, optimization by the compiler etc.
- Although important, we will ignore these technological issues in our analysis.
- The running time depends upon the input size e.g. n .
- Different inputs of the same size may result in different running time. (also depend on structure of input)

Worst-Case Time

- is the maximum running time over all (legal) inputs of size n .
- Let I denote an input instance, let $|I|$ denote its length, and let $T(I)$ denote the running time of the algorithm on input I .
- Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

Problems with $T(n)$

- $T(n)$ is difficult to calculate
- $T(n)$ is also not very meaningful as step size is not exactly defined
- $T(n)$ is usually very complicated so we need an approximation of $T(n)$close to $T(n)$.
- This measure of efficiency or approximation of $T(n)$ is called ASYMPTOTIC COMPLEXITY or ASYMPTOTIC ALGORITHM ANALYSIS

Asymptotic complexity studies the efficiency of an algorithm as the input size becomes large

Example

- If $T(n) = 7n + 100$
- What is $T(n)$ for different values of n ???

n	$T(n)$	Comment
1	107	Contributing factor is 100
5	135	Contributing factor is $7n$ and 100
10	170	Contributing factor is $7n$ and 100
100	800	Contribution of 100 is small
1000	7100	Contributing factor is $7n$
10000	70100	Contributing factor is $7n$
10^6	7000100	What is the contributing factor????

When approximating $T(n)$ we can IGNORE the 100 term for very large value of n and say that $T(n)$ can be approximated by $7(n)$

Example 2

- $T(n) = n^2 + 100n + \log_{10} n + 1000$

n	T(n)	n ²		100n		log ₁₀ n		1000	
		Val	%	Val	%	Val	%	Val	%
1	1101	1	0.1%	100	9.1%	0	0%	1000	90.8%
10	2101	100	5.8%	1000	47.6%	1	0.05%	1000	47.6%
100	21002	10000	47.6%	10000	47.6%	2	0.99%	1000	4.76%
10 ⁵	10,010,001,005	10 ¹⁰	99.9%	10 ⁷	.099%	5	0.0%	1000	0.00%

When approximating T(n) we can IGNORE the last 3 terms and say that T(n) can be approximated by n²

- $\Theta(1)$: Constant time; can't beat it!
- $\Theta(\log n)$: Inserting into a balanced binary tree; time to find an item in a sorted array of length n using binary search.
- $\Theta(n)$: About the fastest that an algorithm can run.
- $\Theta(n \log n)$: Best sorting algorithms.
- $\Theta(n^2)$, $\Theta(n^3)$: Polynomial time. These running times are acceptable when the exponent of n is small or n is not too large, e.g., $n \leq 1000$.
- $\Theta(2^n)$, $\Theta(3^n)$: Exponential time. Acceptable only if n is small, e.g., $n \leq 50$.
- $\Theta(n!)$, $\Theta(n^n)$: Acceptable only for really small n , e.g. $n \leq 20$.

Complexity analysis: examples

Let us assume that $f()$ has complexity $O(1)$

```
for (int i = 0; i < n; ++i) f();
```

 $\longrightarrow O(n)$

```
for (int i = 0; i < n; ++i)  
  for (int j = 0; j < n; ++j) f();
```

 $\longrightarrow O(n^2)$

```
for (int i = 0; i < n; ++i)  
  for (int j = 0; j <= i; ++j) f();
```

 $\longrightarrow O(n^2)$

```
for (int i = 0; i < n; ++i)  
  for (int j = 0; j < n; ++j)  
    for (int k = 0; k < n; ++k) f();
```

 $\longrightarrow O(n^3)$

```
for (int i = 0; i < m; ++i)  
  for (int j = 0; j < n; ++j)  
    for (int k = 0; k < p; ++k) f();
```

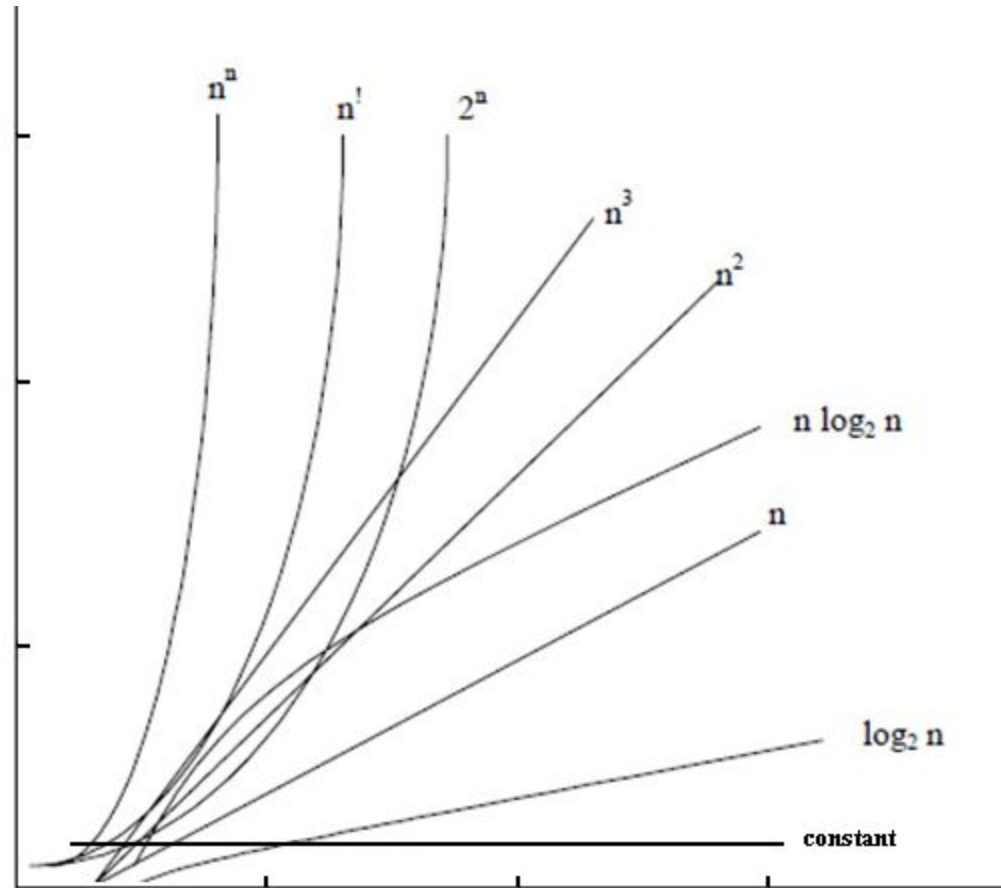
 $\longrightarrow O(mnp)$

```
for (int k = 0; k < b; ++k) f();  
for (int j = 0; j < n; ++j)  
  for (int i = 0; i < m; ++i)
```

 $\longrightarrow O(mnb)$

Growth rates

Which growth rate is better???



Graph from Adam Drozdek's
book

Comparison of Growth Rates

N	$\log_2 N$	$N \log_2 N$	N^2	N^3	2^N
1	0	0	1	1	2
2	1	2	4	8	4
8	3	24	64	512	256
64	6	384	4096	262,144	18446744073709551616
128	7	896	16,384	2,097,152	Approx 6 billion years, 600,000 times more than age of univ.

(If one operation takes 10^{-11}
seconds)

Big O??

Running Time: $O(1)+O(n)+O(n)= O(n)$

```
void print(int * array, int size){
    cout<<"Array:  ";
    for(int i = 0; i < size; i++)
        cout <<array[i]<<" ";
    cout<<endl;
}

int smallest(int array[], int size, int start){
    if(start == size-1)
        return start;
    else
    {
        int small = start;
        for(int i = start+1; i <size; i++){
            if(array[small]>array[i])
                small= i;
        }
        return small;
    }
}

void main(){
    const int n= 5;
    int array[n]={3,5,9,6,1};
    cout<<"Smallest: "<<array[smallest(array,n,0)]<<endl;
    print(array,n);
}
```

O(n) (points to the `print` function)

O(1) (points to the `if` statement in `smallest`)

O(n) (points to the `for` loop in `smallest`)

O(1) (points to the `const int n= 5;` line)

O(n) (points to the `int array[n]={3,5,9,6,1};` line)

O(n) (points to the `print(array,n);` line)

Big-O?

$O(n^2)$

$O(n)$

```
void sortArray(int * a, int size){  
    for(int i = 0;i<size;i++){  
         $O(n)$  ← int small = smallest(a,size,i);  
        int temp = a[i];  
         $O(1)$  a[i] = a[small];  
        a[small] = temp;  
    }  
}
```

```
void main(){  
    const int n= 5;  
    int array[n]={3,5,9,6,1};  
    sortArray(array,n);  
    print(array,n);  
}
```

Running Time: $O(1)+O(n^2)+O(n)= O(n^2)$

Big-O?

$O(n^3)$
)

{

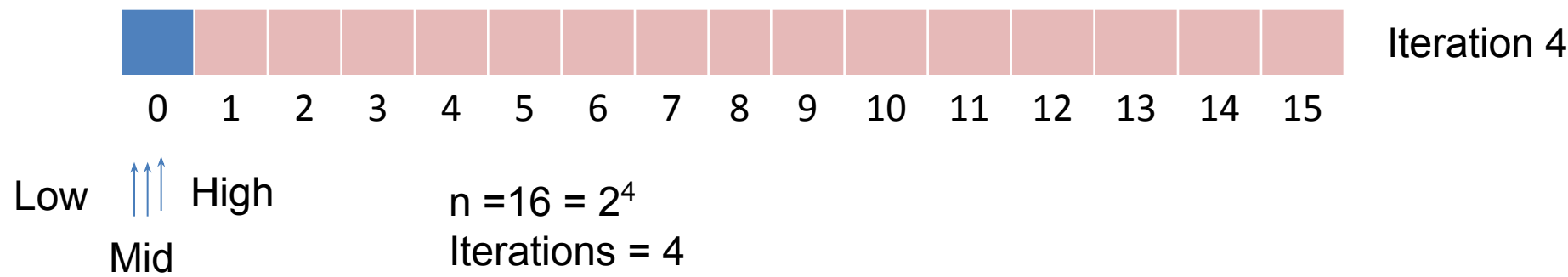
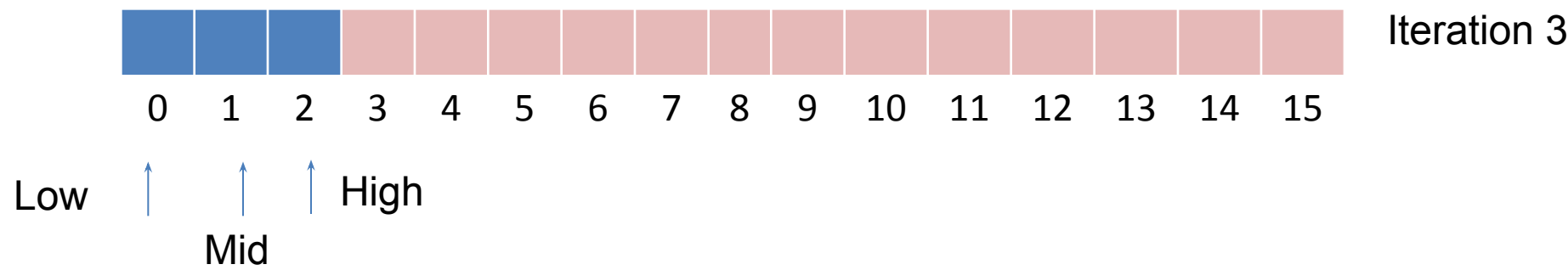
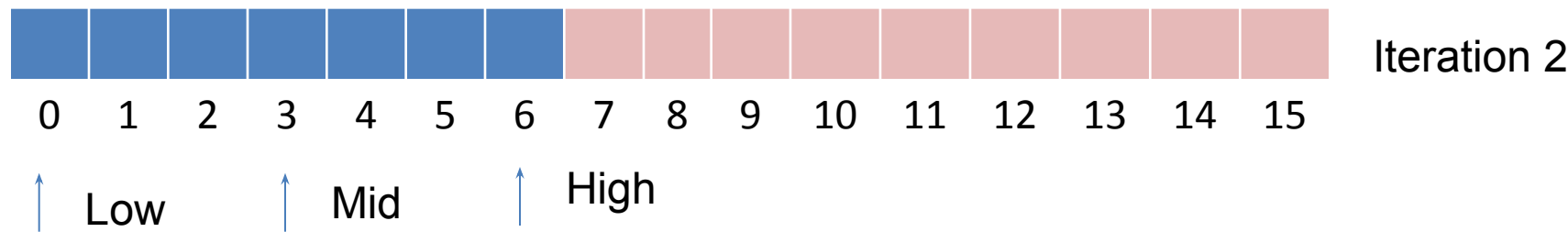
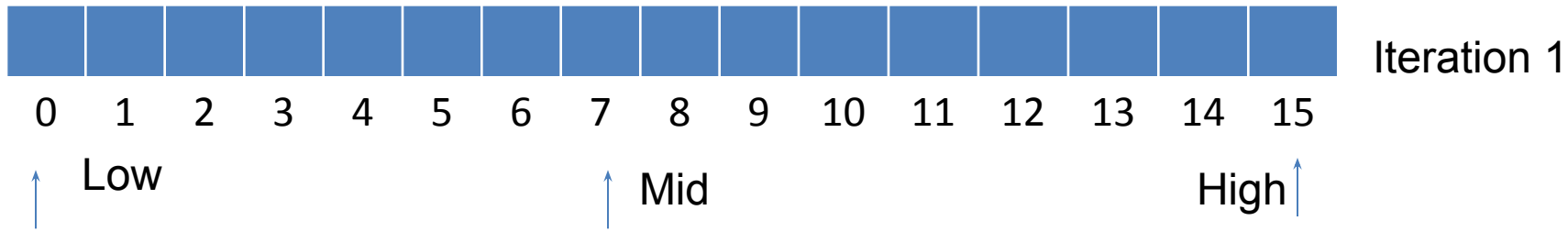
```
void main(){  
    const int n= 5;  
    int array[n]={3,5,9,6,1};  
    for(int i =0; i <n; i++)  
        sortArray(array,n);  
    print(array,n);  
}
```

$O(n^2)$

Running Time: $O(1) + O(n^3) = O(n^3)$

Big-O?

```
bool BinarySearch(int * a,int size, int search){
    int high = size-1;
    int low = 0;
    int mid ;
    while (low<=high){
        mid = (high+low)/2;
        if(a[mid]==search)
            return true;
        else if(search < a[mid]){
            high = mid-1;
        }
        else
            low = mid+1;
    }
    return false;
}
```



$n = 16 = 2^4$
Iterations = 4
 $\log_2 2^4 = 4$ $O(\log_2 n)$